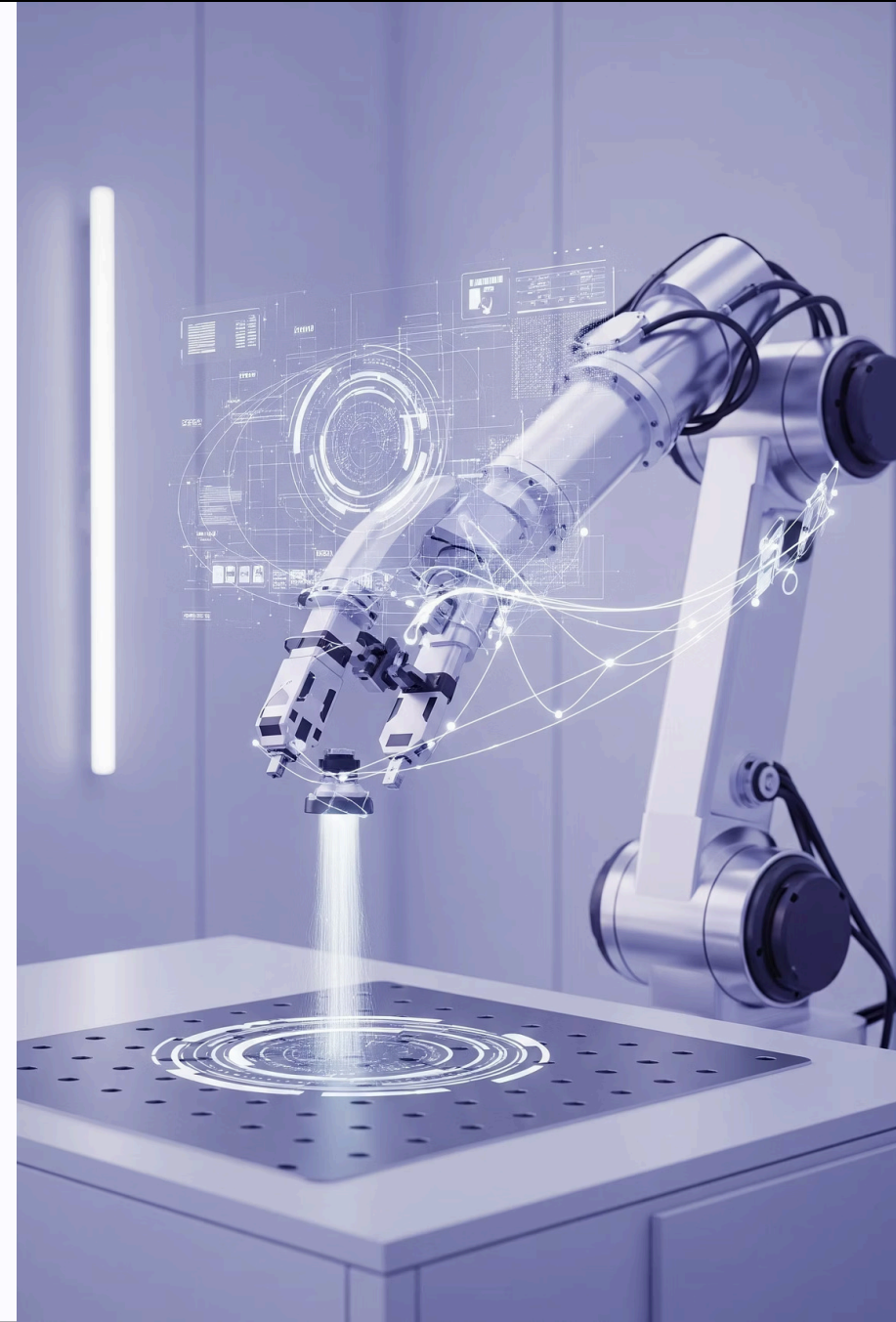


# Real-Time Serving of RL Agents

## From Training Pipelines to Low-Latency Action Loops

This presentation explores how to design and implement systems that serve reinforcement learning agents with millisecond-level responsiveness for mission-critical applications.



# Why Real-Time Serving Matters

Reinforcement learning agents increasingly control **interactive systems** that require rapid response times:

## Speed Requirements

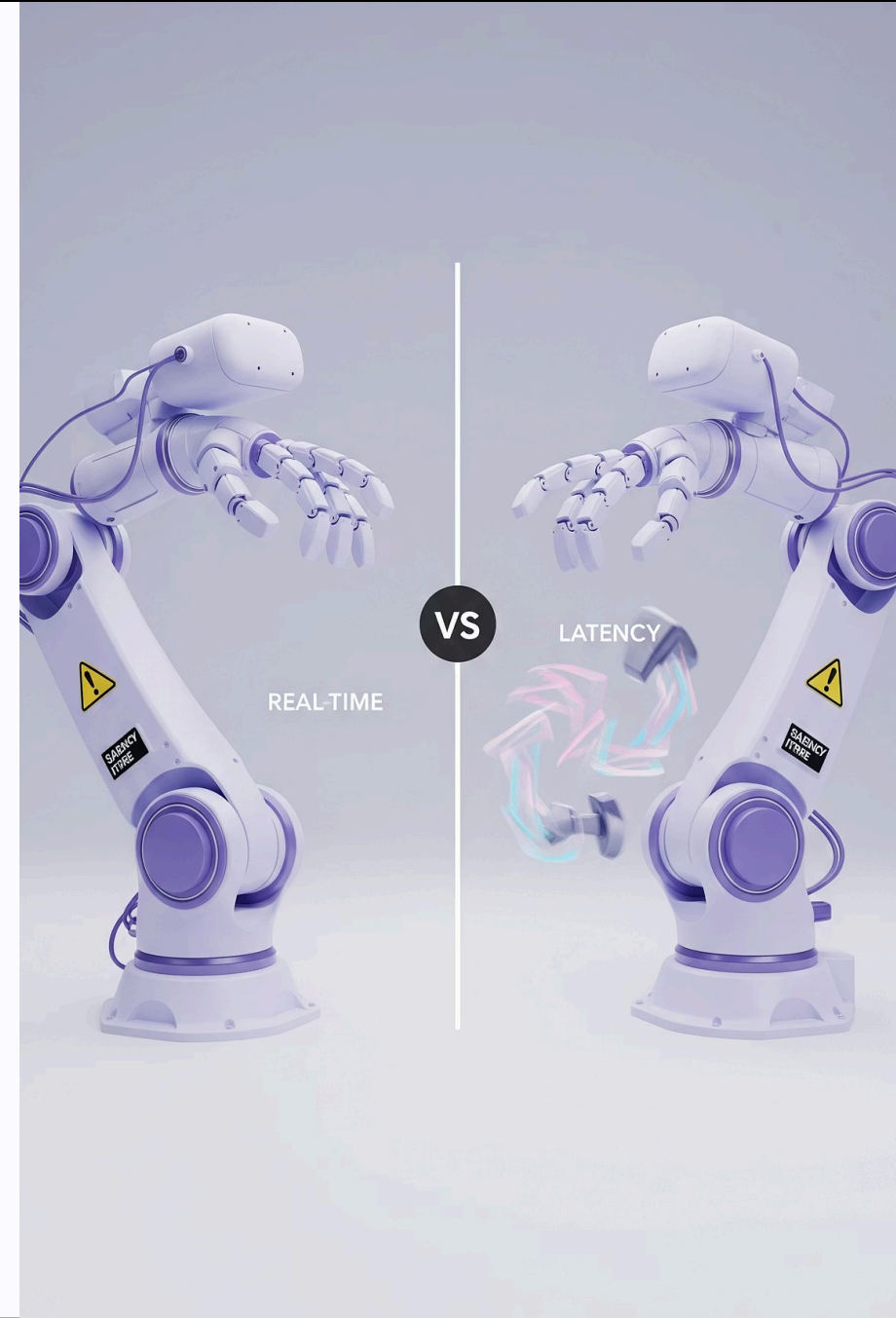
Agents must act in **milliseconds** (10–1000 Hz control loops)

## Consequences

Latency spikes lead to unsafe actions or missed opportunities

## Infrastructure Need

Serving systems must ensure deterministic, low-latency inference



# Core Serving Pipeline

A complete RL serving pipeline involves multiple stages that must work in harmony:



Each stage must complete within the target control cycle timeframe

# Latency Challenges

## Computational Constraints

Model forward pass must complete within control cycle (often  $< 2\text{ms}$ )

## Data-Intensive Preprocessing

High-dimensional observations (images, LiDAR) require heavy preprocessing

## Network Bottlenecks

Cloud deployments introduce variable network delays

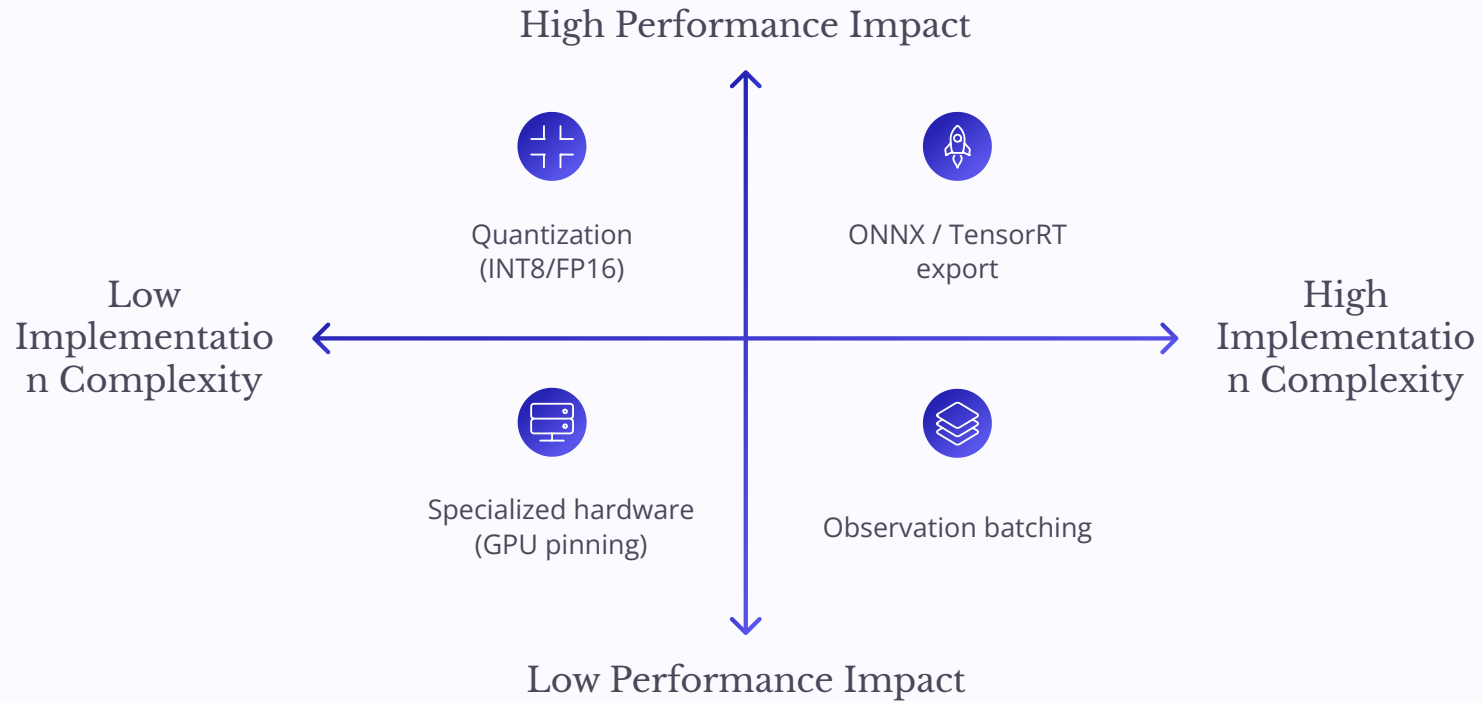
## Jitter

Inconsistent timing leads to unstable control behavior and unsafe actions



Timing violations can cause catastrophic control failures in physical systems

# Optimization Techniques



## Model Optimizations

- Quantization (INT8, FP16) reduces memory footprint and compute time
- ONNX/TensorRT export enables hardware-specific optimizations

## Deployment Optimizations

- Batch observations when safe (for multi-agent systems)
- Use GPU pinning or dedicated edge accelerators (Jetson, Coral)

# Infrastructure Patterns

## On-Device Inference

Lowest latency (<5ms)

Limited compute resources

Ideal for critical control loops

## Edge Servers

Near real-time (5-20ms)

GPU/TPU acceleration

Good balance of power and latency

## Cloud Serving

Higher latency (20-100ms)

Massive scaling capability

Best for non-critical tasks

Hybrid approaches combining edge inference with cloud monitoring are increasingly popular

# Monitoring & Safety

## Critical Metrics

### Performance Metrics

- Inference latency (P50, P99)
- Action frequency
- Real-time reward proxies

### Safety Indicators

- Action saturation detection
- Environment drift
- OOD observation detection



Implement safety wrappers to provide [guardrails](#) that prevent catastrophic actions:

- Action clipping to safe ranges
- Fallback controllers for OOD situations
- Automated circuit breakers when metrics exceed thresholds

# Real-World Use Cases



## Robotics

Robotic arms requiring 1000Hz control loops

Drones with real-time trajectory optimization

Autonomous vehicles with sensor fusion



## Finance

Algorithmic trading with sub-millisecond decisions

Real-time risk assessment

Market-making systems



## Gaming/Simulation

Multi-agent competitive bots

NPC behavior control

Physics-based simulations



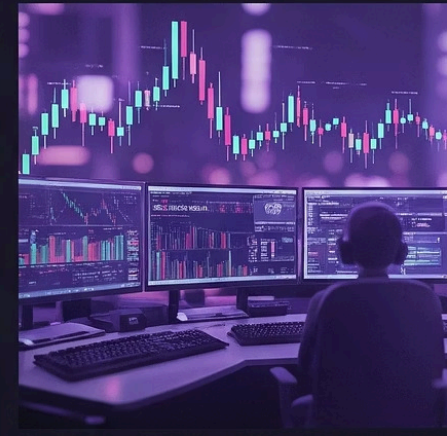
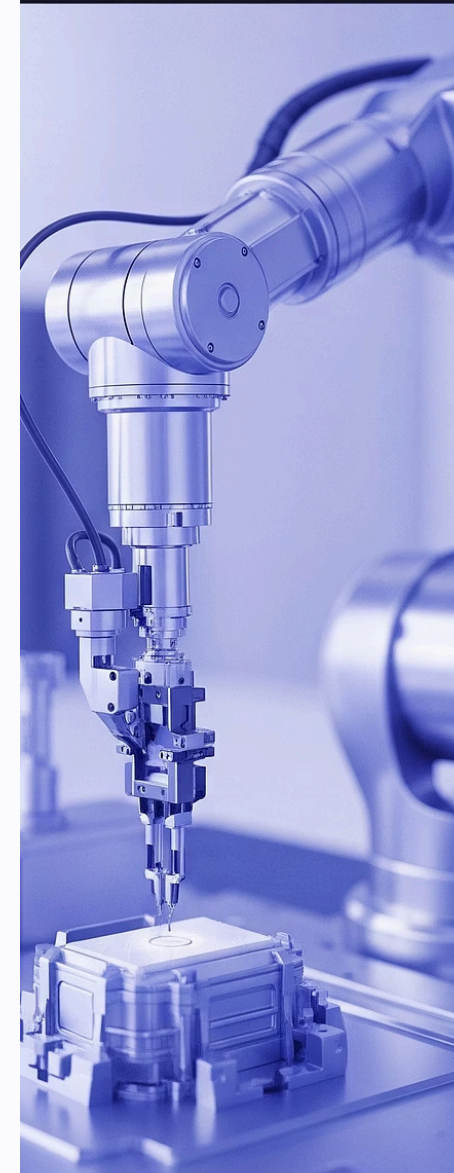
## Industrial Ops

Process control for manufacturing

Predictive maintenance systems

Energy grid optimization

# Reinforcement Learning RL



# Best Practices



## Co-locate agents with environment

Minimize round-trip time between sensors and actuators



## Profile end-to-end latency

Measure complete sensor → inference → action pipeline



## Implement fallback policies

Have simple, safe controllers ready for mission-critical systems



## Log continuously for retraining

Capture full episodes with state, action, reward triplets



## Technical Deep Dive

Consider using specialized hardware threading models:

- Inference on dedicated CPU cores
- NUMA-aware memory allocation
- Real-time scheduling priorities
- Lock-free communication between pipeline stages



## Key Takeaways

# Real-time RL Serving Demands Special Attention

### End-to-End Pipeline

RL serving = real-time **sense** → **infer** → **act** loop that must operate within strict timing constraints

### Latency Impacts

Latency & jitter directly affect safety & performance of RL systems, requiring comprehensive optimization

### Deployment Strategy

Edge/accelerator deployment is key for control tasks requiring sub-10ms response times

### Continuous Improvement

Monitoring + retraining keeps agents reliable over time as environments change

Questions? We're happy to provide more detailed implementation guidance for your specific use case.